Contents

Syllabus **1**
ENPM663 Building AI-Augmented Robotic Systems . . . 1
Course Description . . . 1
Learning Outcomes . . . 2
Required Resources . . . 2
Prerequisites . . . 3
Course Structure . . . 3
Team Formation . . . 3
Major Assignments . . . 4
Team Project Descriptions . . . 4
Grading Structure . . . 5
Peer-Review Contribution (15% of grade) . . . 6
Academic Integrity . . . 7
Tips for Success in this Course . . . 8
Policies and Resources for Graduate Courses . . . 8
Names, Pronouns, and Self-Identifications . . . 9
Communication with Instructor . . . 9
Course Outline . . . 9
Resources and Accommodations . . . 10
Participation . . . 11
Course Evaluation . . . 11
Copyright Notice . . . 11

Syllabus

ENPM663 Building AI-Augmented Robotic Systems

Spring 2027

Field	Value	Field	Value
Section	0101 & RO01	**Professor**	Z. Kootbally
Credits	3	**Pronouns**	He/Him
Course Dates	TODO: verify Spring 2027 dates	**Email**	zeidk@umd.edu
Course Times	TODO: verify weekly meeting time	**Office Hours**	By appointment
Classroom	TODO: assigned room		

Course Description

This hands-on course examines how modern AI techniques, including large language models (LLMs) and vision-language models (VLMs), are integrated into ROS 2 based robotic systems. Students study classical task and motion planning, then add AI-augmented counterparts at the task-planning

layer: LLM agents for task planning (the observe-act-observe loop, tool use, and structured output), VLM grounding for scene understanding, and closed-loop replanning. The end-to-end vision-language-action (VLA) frontier is surveyed for context, but motion planning, grasp execution, and control remain classical (MoveIt2); this course teaches the modular, auditable stack rather than training learned policies. The course is structured around the Agile Robotics for Industrial Automation Competition (ARIAC) 2025 (https://pages.nist.gov/ARIAC_docs/en/latest/) as a unifying testbed, but the concepts and engineering patterns taught are general to autonomous robotic systems.

Students entering this course are expected to have working knowledge of ROS 2 (publishers and subscribers, services, actions, parameters, launch files). Week 1 provides a refresher on the ROS 2 topics most directly used in the course (lifecycle nodes, TF2, QoS, executors). A significant amount of Python programming is required, with selected use of C++ for components where it is the industry standard.

All course work is team-based. There are no individual assignments. See the Team Formation and Peer-Review Contribution sections for details.

Learning Outcomes

After successfully completing this course you will be able to:

- Design system architectures that integrate classical robotic components (perception, planning, control) with AI-based components (LLMs, VLMs).
- Apply knowledge-representation techniques (frames, ontologies, hierarchical task and world models) to design symbolic representations of ARIAC scenes and tasks that are consumable by both classical planners (PDDL) and AI planners (LLM/VLM).
- Implement behavior trees in `py_trees_ros` for robust task execution, including failure handling and recovery.
- Apply classical task-level planning using PDDL to generate executable plans for robotic systems.
- Build LLM agents for task planning that reason in an observe-act-observe loop, produce structured plans and tool calls via prompting and JSON-schema validation, and invoke robot actions through tool/function calling (MCP).
- Use vision-language models to ground robotic plans in observed scenes and recover from perception failures.
- Design robust execution patterns (failure handling, recovery, replanning) for AI-driven robotic systems where LLM and VLM outputs may be incorrect.
- Plan and execute robot motion using MoveIt2 for pick-and-place and assembly tasks.
- Evaluate AI-augmented robotic systems using quantitative metrics for plan validity, grounding accuracy, execution success, and end-to-end task performance.

Required Resources

- **Course Website:** elms.umd.edu
- **Operating system:** Ubuntu 24.04.x LTS (Noble Numbat): https://releases.ubuntu.com/noble/
- **Middleware:** ROS 2 Jazzy Jalisco: https://docs.ros.org/en/jazzy/Installation.html

- **Simulator:** Gazebo Harmonic: https://gazebosim.org/docs/harmonic/
- **Editor:** Visual Studio Code: https://code.visualstudio.com/
- **ARIAC 2025:** https://pages.nist.gov/ARIAC_docs/en/latest/
- **AI access:** Either a paid LLM API account (e.g., Anthropic Claude, OpenAI, Google Gemini) or local open-weight models (e.g., recent Llama or Qwen releases). Course team projects will accept either; cost guidance will be provided per project.

Prerequisites

Working knowledge of ROS 2 is required. Specifically, students should be comfortable with:

- Creating and managing ROS 2 packages and workspaces.
- Writing publishers, subscribers, services, and action clients and servers in Python.
- Using launch files, parameters, and basic command-line tools (`ros2 topic`, `ros2 node`, etc.).
- Reading C++ ROS 2 code (writing C++ is not a prerequisite, but reading it is expected).

A diagnostic quiz in Week 1 helps students and the instructor calibrate on ROS 2 background. The quiz is ungraded.

Course Structure

- Before class meetings, students need to consolidate the previous lessons since each week builds directly on the previous one. Early in the course, expect roughly 4 hours per week on team project work. As the course progresses, projects may require 6 to 8 hours per week per student.
- Quizzes are administered in class (unless you are registered as a remote student). Absent students cannot take quizzes.
- Class meetings combine lecture and in-class exercises. Bring a laptop to every meeting. Class attendance and active participation are not graded but are strongly encouraged.
- Late submissions are penalized 10% of the team project's grade per calendar day late, up to 3 days; submissions more than 3 days late receive no credit. Extensions are granted only when more than one team needs additional time on a submission.
- No quizzes or team projects will be dropped. PT5 (the capstone) is required; there is no alternative to PT5's grade.
- PT5 is evaluated on an unseen ARIAC scenario. Teams run their integrated system in class, and the performance is graded.
- A PT5 presentation is required at the end of the semester. Teams discuss their approach, design decisions, and results.
- All team projects use a peer-review system. Each team member rates the contributions of other team members after each project. See Peer-Review Contribution.

Team Formation

- All course work is team-based across 5 team projects (PT1 through PT5). There are no individual assignments.

- **Team size: 3 students.** Teams of 2 are allowed only with instructor approval (e.g., when cohort numbers do not divide evenly or for remote students). Teams of 4 or more are not permitted; integration projects of this scope typically generate coordination overhead without productivity gain past three members.
- **Teams are formed in Week 3**, after the diagnostic quiz and the Week 2 lecture on system architectures. This gives students time to meet classmates and self-organize.
- Teams stay together across all 5 PTs. Stable team composition lets the peer-review signal accumulate across the semester and avoids losing team-formation cost between projects.
- A team change request requires a written rationale and instructor approval, and is expected only in cases of sustained imbalance documented in peer review.

Major Assignments

- **Team Projects (PT1 through PT5).** Five team-based projects that build the AI-augmented robotic system layer by layer. PT5 is the capstone, evaluated on an unseen ARIAC scenario. Detailed instructions and submission guidelines are provided for each project. See the Course Outline for the schedule.
- **Quizzes.** Quizzes assess your understanding of course material. They are administered at the beginning of class, either on Canvas or on paper, and typically last 10 to 20 minutes. All quizzes are closed-notes.
- **Peer-Review Contribution.** After each team project, every student rates the contribution of their teammates. See Peer-Review Contribution.

Team Project Descriptions

PT1: Behavior Tree for a Simple Kitting Workflow

Hand-out: Week 2 / **Due:** Week 5 / **Weight:** 10%

Build a runnable `py_trees_ros` behavior tree that executes a fixed, single-order kitting sequence with hard-coded part poses. The focus is BT structure (sequences, selectors, decorators, blackboards) and ROS 2 action-client leaves that call ARIAC's pick and place skills as opaque actions. No perception or planner integration yet; this PT is the "I understand BTs" gate.

Deliverable: ROS 2 package with a launchable BT, plus a short write-up (1 to 2 pages) including the BT diagram and the rationale for the composite choices.

PT2: Perception Pipeline

Hand-out: Week 5 / **Due:** Week 7 / **Weight:** 10%

Detect ARIAC parts in camera images and publish their 3D poses to a TF-aware ROS 2 topic; then wire PT1's BT to consume real perception output instead of hard-coded poses. ARIAC context: overhead RGB-D cameras above the supply bins, with ARIAC's part types (batteries, sensors, regulators, pumps) in their color variants.

Deliverable: Perception package, comparison of detected vs. ground-truth poses on at least 5 ARIAC scenes with error metrics, and the updated PT1 BT that consumes the perception topic.

PT3: PDDL Planner + BT Executor (Classical Baseline)

Hand-out: Week 7 / **Due:** Week 10 / **Weight:** 10%

Replace PT1's hard-coded action sequence with a PDDL planner that generates plans from a goal; execute those plans through a BT pattern that translates each PDDL action into a BT subtree. ARIAC context: multi-part kitting orders with varying contents read from ARIAC's order interface, including at least one scenario with a faulty part that triggers re-planning. Spans Spring Break.

Deliverable: PDDL domain and problem files, planner integration (Fast Downward or Unified Planning Framework), BT executor that consumes plans, demo on 2 to 3 ARIAC kitting scenarios (one of which triggers a re-plan), and a brief reflection on what PDDL captures well vs. where it strains.

PT4: LLM Agent for Task Planning with VLM Grounding (Extends PT3)

Hand-out: Week 10 / **Due:** Week 13 / **Weight:** 10%

Swap the PDDL planner for an LLM agent that consumes a VLM-grounded scene description and the order, and that outputs a structured plan (JSON schema) or per-step tool calls which the same BT executor from PT3 runs unchanged, replanning in a closed loop when execution drifts from the expected state. ARIAC context: same kitting scenarios as PT3, plus at least one scenario PT3's PDDL domain cannot handle without code changes (e.g., a new part type or an unusual ordering constraint). The LLM should adapt at prompt time; PDDL would need a domain rewrite.

Deliverable: LLM client module, VLM grounding module, executor modifications, side-by-side comparison of PDDL vs. LLM plans on identical inputs (validity, plan length, cost), demo, and a reflection on LLM failure modes and how they were caught.

PT5: Integrated Capstone on Unseen ARIAC Scenario

Hand-out: Week 12 / **Due:** Week 15 / **Weight:** 20%

PT5 combines selected ARIAC challenges with scenarios drawn from PT1 through PT4. Teams integrate their existing PT1 to PT4 components (behavior tree, perception pipeline, PDDL planner, LLM planner with VLM grounding) and apply the recovery and robustness patterns introduced in Week 12 to handle a scenario that includes ARIAC-specific challenges (e.g., faulty parts, faulty grippers, sensor blackouts, kit rearrangement, high-priority orders) that were not present in any single earlier PT. Manipulation uses classical MoveIt2 with parameterized grasps; AI complexity lives in task planning, scene grounding, and replanning. Specific challenge selection is published at PT5 hand-out (Week 12); the exact ARIAC scenario used for grading is unseen by teams until the live demo in Week 15.

Deliverable: Integrated codebase, live demo in class (graded on performance against the unseen scenario), short presentation (5 to 7 minutes) covering architecture, design decisions, failure modes encountered, and how they were handled.

Grading Structure

Component	Percentage
Team Projects (PT1 through PT5)	60%
Quizzes	25%
Peer-Review Contribution	15%
Total	**100%**

Team project weights

Team Project	Weight	Notes
PT1	10%	
PT2	10%	
PT3	10%	
PT4	10%	
PT5 (capstone)	20%	Evaluated on an unseen ARIAC scenario, demoed live in Week 15.

Final Grade Cutoffs

	+		+		+		+	
	97.00%		87.00%		77.00%		67.00%	
A	94.00%	B	84.00%	C	74.00%	D	64.00%	F < 60.00%
	-		-		-		-	
	90.00%		80.00%		70.00%		60.00%	

Final letter grades are assigned based on the percentage of total assessment points earned. Standards are applied consistently: being close to a cutoff is not the same as making the cut (89.99 is not 90.00).

Peer-Review Contribution (15% of grade)

The Peer-Review Contribution category is assessed *by teammates*, not by the instructor, and reflects intra-team contribution. It is separate from class participation, which is ungraded.

Instrument

After each of the 5 team projects, every student completes a peer-review form via ELMS-Canvas, rating each teammate (not themselves) on the following five dimensions:

1. **Technical contribution.** Did this teammate contribute substantively to the technical work (code, design decisions, debugging, system integration)?
2. **Reliability and communication.** Did this teammate show up to meetings, respond to messages, and deliver agreed work on time?

3. **Quality of work.** Was their contribution of a quality you could build on (clean, complete, tested code), or did you need to redo it?
4. **Knowledge and skill.** Did this teammate bring relevant knowledge or actively work to acquire what was needed?
5. **Team-player behavior.** Did this teammate help others, listen, accept feedback, and contribute to a constructive team environment?

Each dimension uses the following 5-point scale:

- **5 = Consistently outstanding.**
- **4 = Above expectations.**
- **3 = Meets expectations.**
- **2 = Below expectations; needs improvement.**
- **1 = Substantially below expectations; raises concern.**

A short written comment (one to three sentences) is required for each teammate. Comments are mandatory whenever a dimension is rated 2 or below, or 5. Uniform ratings (all 3s or all 5s for every teammate with no written justification) will be returned ungraded and must be redone.

Confidential concern flag

Each form has a confidential checkbox: "I want the instructor to follow up about team dynamics on this project." Checking it triggers a private check-in with the instructor. Teammates are not told the flag was raised.

How the score is computed

- For each PT, a student's peer score is the mean of their teammates' ratings across all five dimensions, normalized to a 0 to 100 scale (a mean of 3.0 maps to 80; the scale rewards above-expectations contribution while protecting against single- teammate penalties).
- The PT-level peer scores are averaged across all 5 PTs (equal weight) and multiplied by 15% to produce the final Peer-Review Contribution grade.
- Outliers are reviewed by the instructor before being applied. If one teammate scores a peer 1 across all dimensions while another scores them 4, the instructor reads the written comments and decides whether to include both ratings, discard the outlier, or initiate a team conversation.

Protections against gaming

- Ratings are visible to the instructor only, never to the rated student.
- A single low rating from one teammate cannot, by itself, push a student below 70 in this category; any team-averaged peer score under 70 triggers instructor review before being assigned.
- The instructor reserves the right to override peer-assigned scores in documented cases of reciprocal rating (you rate me high if I rate you high) or coordinated retaliation.

Academic Integrity

Some submissions will be collected via Turnitin on the course ELMS page to support scholarly writing and to verify the integrity of student work.

The University Code of Academic Integrity is designed to ensure that the principles of academic honesty and integrity are upheld. The University of Maryland does not tolerate academic dishonesty. All acts of academic dishonesty will be dealt with in accordance with the provisions of this code.

Course assistance websites (such as CourseHero) and AI-generated content are not permitted as sources unless the instructor explicitly grants permission. Specifically:

- Using AI tools (including LLM APIs) is required for some team projects (notably PT4 and PT5). The expectation is that the team understands, designs, and debugs the code that calls these tools. Submitting AI-generated code that the team cannot explain is an academic integrity violation.
- Teams may use AI tools to clarify concepts or look up syntax for any team project, but the submitted code and report must be the team's own work, and any team member must be able to explain any portion of it on request.
- For quizzes, no external tools or notes are permitted.

On each team project submission, every team member must sign the following pledge: *"I pledge on my honor that I have not given or received any unauthorized assistance on this team project."* On each quiz, the same pledge applies to the individual student.

Tips for Success in this Course

1. **Participate.** Engage with the lectures, ask questions, and discuss course content with your teammates and classmates. Robotic systems are integration problems; you will learn faster by talking through design decisions with peers.
2. **Manage your time.** The course moves quickly. Block out time in your schedule for team project work and include extra time for environment and dependency issues.
3. **Coordinate with your team.** Set a recurring meeting time. Use a shared repository from day one. Agree early on who owns which subsystem.
4. **Log in regularly.** Check ELMS-Canvas several times a week for announcements and discussion replies. Check daily near submission deadlines.
5. **Do not fall behind.** Each week builds on the previous one. If you feel you or your team are falling behind, contact the instructor early.
6. **Use ELMS notification settings.** Enable instant or daily notifications so you do not miss announcements.
7. **Ask for help.** For ELMS or technology issues, contact IT support. For course content, contact the instructor or classmates.

Policies and Resources for Graduate Courses

It is our shared responsibility to know and abide by the University of Maryland policies that apply to all courses, including:

- Academic integrity
- Student and instructor conduct
- Accessibility and accommodations
- Attendance and excused absences

- Grades and appeals
- Copyright and intellectual property

See https://gradschool.umd.edu/course-related-policies for the full set of graduate course-related policies.

Names, Pronouns, and Self-Identifications

The University of Maryland recognizes the importance of a diverse student body and is committed to fostering inclusive and equitable classroom environments. You are invited, if you wish, to tell the instructor how you want to be referred to in this class, both in terms of your name and pronouns. The pronouns someone uses are not necessarily indicative of their gender identity. Visit lgbtq.umd.edu to learn more.

Whether and how to disclose how you identify in terms of gender, race, class, sexuality, religion, ability, and other aspects of your identity is your choice and should be self-identified, not presumed or imposed.

Communication with Instructor

- **Email.** Please email zeidk@umd.edu for personal, academic, or intellectual concerns. Please do not email about questions that are easily answered by the syllabus or ELMS (e.g., due dates, point values). Responses are typically sent within 24 hours during weekdays.
- **ELMS.** Important announcements (including project changes and due-date changes) are posted on ELMS. Enable email and announcement notifications to avoid missing messages.
- **Communication with peers.** With a diversity of perspectives and experience, disagreement and debate are expected. Conduct yourself professionally and respectfully. This course is not a space for hate speech, harassment, or derogatory language. Alert the instructor immediately if you feel threatened, dismissed, or silenced.

Course Outline

Week	Topic	Deliverables
1 (TODO: date)	Course Overview and ROS 2 Refresher	Diagnostic quiz (ungraded)
2 (TODO: date)	System Architectures for AI-Augmented Robotics	PT1 handed out
3 (TODO: date)	Knowledge Representation for AI-Augmented Robotics	Teams formed
4 (TODO: date)	Behavior Trees for Task Execution	Quiz #1
5 (TODO: date)	Sensing and Perception for Robotic Systems	Quiz #2; PT1 due; PT2 handed out

Week	Topic	Deliverables
6 (TODO: date)	Task-Level Planning with PDDL	–
7 (TODO: date)	Plan Execution Patterns with Behavior Trees	Quiz #3; PT2 due; PT3 handed out
8 (TODO: date)	SPRING BREAK	–
9 (TODO: date)	LLM-Based Task Planning I: Agent Loops, Tool Use, and Structured Output	–
10 (TODO: date)	LLM-Based Task Planning II: VLM Grounding and Agentic Replanning	Quiz #4; PT3 due; PT4 handed out
11 (TODO: date)	Frames, Transforms, and Motion Planning	–
12 (TODO: date)	Recovery, Robustness, and Failure Handling for AI-Driven Systems	Quiz #5; PT5 handed out
13 (TODO: date)	Evaluation and Measurement Science for AI-Augmented Systems	PT4 due
14 (TODO: date)	PT5 Lab	–
15 (TODO: date)	PT5 Demo and Presentation	PT5 due

Note: This is a tentative schedule and subject to change as necessary. Monitor the course ELMS page for current deadlines. In the event of a prolonged university closing or extended absence, adjustments to the schedule, deadlines, and assignments will be made based on the duration of the closing and the specific dates missed.

Resources and Accommodations

Accessibility and Disability Services

The University of Maryland is committed to creating and maintaining a welcoming and inclusive educational environment. The Accessibility & Disability Service (ADS) provides reasonable accommodations to qualified individuals. ADS cannot assist retroactively, so it is generally best to request accommodations several weeks before the semester begins or as soon as a disability becomes known. Any student who needs accommodations should contact the instructor as soon as possible.

For assistance, contact ADS at 301-314-7682 or adsfrontdesk@umd.edu.

Student Resources and Services

UMD provides a wide range of student support resources. See https://studentsuccess.umd.edu/ for the full list, including the Writing Center, the Counseling Center, and academic support services.

Notice of Mandatory Reporting

As a faculty member, the instructor is designated as a Responsible University Employee and must report all disclosures of sexual assault, sexual harassment, interpersonal violence, and stalking to UMD's Title IX Coordinator.

For confidential support, contact CARE to Stop Violence at 301-741-3442 or the Counseling Center at 301-314-7651.

You may also contact UMD's Title IX Coordinator at 301-405-1142 or titleIXcoordinator@umd.edu.

Basic Needs Security

If you have difficulty affording groceries or accessing sufficient food, or lack a safe and stable place to live, please contact UMD's Division of Student Affairs for information about available resources. Let the instructor know if there is any way the course can help.

Veteran Resources

UMD provides additional support for student veterans through the Office of Veteran Student Life and the Counseling Center. Veterans and active duty military personnel with special circumstances (e.g., deployments, drill requirements, disabilities) are encouraged to communicate these to the instructor in advance.

Participation

Given the hands-on nature of this course, attendance is important to note-taking and your performance. Class discussion is a critical component of learning. Each student is expected to make substantive contributions to in-class exercises and to their team's work. Students with a legitimate reason to miss a session should communicate in advance with the instructor, except in case of emergency. Students who miss a session are responsible for learning what they missed.

Class attendance and class-discussion participation are not directly graded. Intra- team contribution is graded via the Peer-Review Contribution category.

Course Evaluation

Please submit a course evaluation through Student Feedback on Course Experiences at the end of the semester. All information submitted is confidential.

Copyright Notice

Course materials are copyrighted and may not be reproduced for anything other than personal use without written permission.